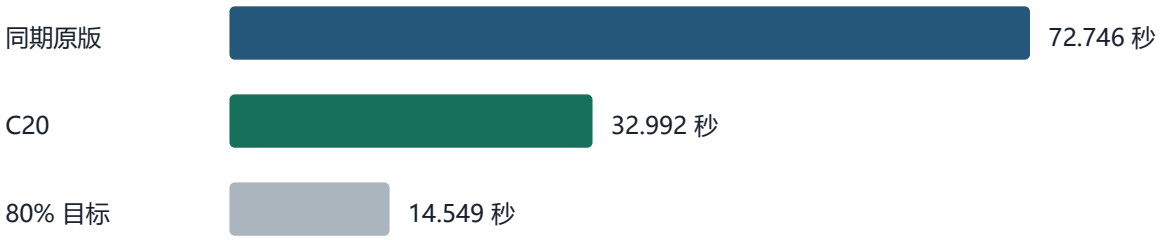
Tight Inclusion CCD C20 版本改动分析报告

报告日期 2026 年 10 月 13 日 | 实验日期 2026 年 10 月 10 日

C20 将全量 CCD 核心耗时中位数从 72.746 秒降至 32.992 秒，降低 54.65%，达到 2.205 倍加速。四组漏检均为 0，误报总数从 312 个减至 191 个。20 个有效候选已经完成，按约定上限停止；后续提高到 80% 的目标未达到。

54.65% 指 CCD 核心计算时间的降幅，不是程序从启动到结束的总耗时降幅。数据读取和解析等开销不计入这项成果。

本版本以原求解器的实现优化为主，同时加入有保守性依据的几何剔除，并优化数值误差界的计算。BFS 区间求根框架仍然保留。报告解释结果如何改善、当前优化覆盖哪些路径，以及结论的适用边界。



三次全量 CCD 核心耗时的中位数　越短越快

全量轮次	原版 CCD 秒	C20 CCD 秒
1	72.745920	32.749760
2	72.927355	32.992082
3	72.412401	33.248423
中位数	72.745920	32.992082

实验条件与统计口径

每轮 59,387,883 条查询；顺序为原版 1、C20 1、原版 2、C20 2、原版 3、C20 3。先求每轮四组时间之和，再取三次总数的中位数。历史 74.37439 秒仅作参考，未作为加速比的分母。

同一台 Intel Core Ultra 7 265KF，Windows 11，MSVC 19.44 Release；单线程绑定逻辑 CPU 0。double、BFS、tolerance=1e-6、max_iterations=1e6、minimum_separation=0、t_max=1、no_zero_toi=false。使用相同逐查询 Timer，计时包含 API 参数转换和 CCD 调用，排除数据读取、缓存转换、比较及写盘。

代码基准 v1.0.6；最佳核心提交 0c1f229b4e10a3ad95033956f6604f5740c64c8d。全部结论对应已保存的同期实验 [S1]，核心与最终实测快照一致 [S2]。

1 分组性能与正确性

四组均有提速。仿真点面相对收益最大；人工查询数量很少，但单条计算成本仍高。VF 表示点面查询，EE 表示边边查询。

测试组	查询数	原版 秒	C20 秒	耗时降低	加速比
仿真点面 VF	17,959,962	12.038	3.999	66.78%	3.010×
仿真边边 EE	41,372,584	28.265	12.271	56.59%	2.303×
人工点面 VF	21,208	7.970	4.576	42.58%	1.742×
人工边边 EE	34,129	24.441	12.076	50.59%	2.024×

表中各组分别取三次中位数；这些中位数不能相加后代替总耗时中位数。分组值由最终六轮原始报告重新计算，计算结果另存于 c20-analysis-metrics.json [S3]。

漏检和误报

测试组	原版误报 FP	C20 误报 FP	C20 漏检 FN	减少的误报
仿真点面 VF	2	2	0	0
仿真边边 EE	17	6	0	11
人工点面 VF	90	53	0	37
人工边边 EE	203	130	0	73
合计	312	191	0	121

三次全量 C20 的上述计数完全一致。误报总数减少 38.78%，各组均未超过原版上限。此百分比描述误报数量变化，不是所有查询的错误率。

碰撞时间的保持方式

每条查询比较 hit、TOI 和 output_tolerance 的全部 24 字节。仅允许原版的真值阴性误报变为 no-hit，且返回正无穷 TOI 和输入容差。每轮只有 121 条符合该例外；其余 59,387,762 条查询输出逐位一致，包括所有保留碰撞的 TOI。

数据集没有精确 TOI 真值，因此这里证明的是没有改变保留碰撞的原版输出行为，未独立测量绝对碰撞时间误差。tolerance=1e-6 是原算法的距离容差，不能解释为绝对时间误差上限。

剩余时间集中在哪里

人工查询共 55,337 条，仅占全量 0.0932%，却占三次 C20 运行时间的 50.08% 至 50.73%。以总耗时居中的第 2 轮看，仿真 EE 占 37.19%，人工 EE 占 36.71%，是当前两个最大的耗时组。

2 当前优化覆盖的计算流程

当前涉及算法判定层面的增强：新增无碰撞证书会提前结束部分查询，改变执行流程；但没有替换 Tight-Inclusion 的 BFS 区间求根框架，也没有设计新的细分策略。因此本版本应归类为实现优化与保守几何过滤增强的组合。

下表按一次查询的执行顺序列出优化位置。便宜的拒绝判定先处理容易排除的查询；较贵的证书仅在搜索已持续到指定计数时调用。未被证书排除的查询继续原区间求解流程。

位置	当前做法	减少的工作	候选来源
VF 入口	先做端点 swept AABB	对明显分离的点面，省去误差界、容差和搜索初始化	C07
误差界准备	8 个顶点指针；向上舍入乘法	省去 vector 分配、坐标复制和通用立方计算	C15 C19
初始参数盒	提前执行原初始盒判定；直接给出 0/1 数组	被拒绝时省去细分容差计算和搜索队列	C02 C05
EE 初筛顺序	原初始盒拒绝之后再做 AABB	减少额外 AABB 在容易拒绝的 EE 上的开销	C16
区间角点评估	复用相同时间和空间插值；精确二进制倒数	减少重复算术与除法	C01 C03 C04
搜索数据与调用	内联、模板回调、紧凑节点、队列容量复用	减少调用、节点复制与反复分配	C06 C10 C11 C13 C20
BFS refine 为 32	先尝试 Bernstein，再尝试投影分离	用整段轨迹无碰撞证明提前结束困难阴性查询	C08 C09 C14
证书无法判定	继续原 BFS 细分和终止逻辑	保留求解能力及保守回退	原框架保留

refine==32 是源码中的有效搜索计数触发点，不表示每个查询都先完成 32 次几何二分。整个 BFS 调用中，这组证书最多尝试一次。短路顺序为 Bernstein 成功即可结束，否则尝试投影证书。

两类收益及其区别

实现优化降低一次运算、一次判定和一次队列操作的成本。几何证书则减少需要执行的后续搜索。二者会相互影响：便宜路径越快，给所有查询都加上昂贵证书越可能得不偿失，因此证书的调用时机本身也是优化内容。

本研究未对最终 C20 做逐项关闭的完整消融。可以确认组合版本的 54.65% 收益，但不能给出每一项对总收益的独立百分比，也不能将各阶段差值简单相加。

3 工程实现上具体减少了什么

重复插值和除法

区间盒有八个角点，但时间只有上下界两个值。原表达式按角点重复求四个顶点的时间插值；专用求值器每个时间值只算一次，再复用相同 u、v 的空间项。按源码表达式计，每坐标轴的四顶点时间插值从 32 份降至 8 份；这不是机器指令计数或独立速度测量。

内部区间分母为 2 的整数次幂，分母及其倒数都能被 double 精确表示。在相同舍入模式下，将已有中间值 x 除以该分母改为乘其精确倒数，结果相同。前一步乘分子的舍入仍保留，未改成合并比例的表达式，也未引入 FMA 重结合。

初筛与输入准备

原来第一次 BFS 才做的初始盒判定前移到容差计算和队列构造之前；判定中的角点、误差界和比较条件保持一致。对 $[0,1]^3$ 初始域，直接构造固定的 0/1 数组，避免一般区间转换。误差界输入改为栈上的八个 const 顶点指针，避免临时 vector 分配和坐标复制。

调用和区间数据

小型整数与区间函数移入头文件内联，split_and_push 的 std::function 回调改为模板参数，让编译器看到实际调用对象。NumCCD 保留完整 uint64 分子和 uint8 指数，仅压缩布局中的对齐填充，不截断数值。

目标编译器上的结构	原版 字节	C20 字节	大小降低
NumCCD	16	9	43.75%
Interval3	96	54	43.75%
BFS 队列节点	104	60	42.31%

搜索队列分配

C20 仍使用原 std::priority_queue 的节点、比较器、入队和出队顺序；改变的是底层 vector 容量的生命周期。每次调用结束清空元素，将不超过 65,536 个节点的容量保留在线程局部存储中，供后续同类型查询复用。

每线程 VF、EE 分别最多保留约 3.75 MiB，共约 7.5 MiB；超出阈值的缓冲区释放。容量限制不限制搜索节点数，不是达到上限就截断搜索。结构布局改变了 ABI，集成时必须重新编译依赖该结构的代码。

4 几何剔除与数值保守性

新增拒绝路径要求得到严格的无碰撞证明。接触边界、含零区间或不确定结果均不能据此拒绝。数值证明与全量验证分别承担不同职责：前者解释一般保守性，后者验证实现和当前数据。详细推导见 [S4]。

端点 swept AABB

线性运动顶点位于其起终点坐标区间内，运动线段或三角形又是顶点的凸组合。因此，两个端点包围盒在任意坐标轴严格分离，就能排除整段运动中的碰撞。实现只用端点 min/max 和严格比较；边界相等时不剔除。该路径仅在 BFS 且 minimum_separation=0 时启用。

Bernstein 共面性证书

点面或边边相交要求四个顶点共面。三个相对线性轨迹的行列式是时间三次多项式，可写为四组系数与非负基函数的组合。用向外舍入区间包围系数；四组系数严格同号时，行列式在整个 [0,1] 上不为零，可以排除碰撞。

输入先扩展到相邻 double，加减乘各步用 nextafter 向外扩展。严格符号无法确定就回到原算法。证书只在 double 路径启用；坐标非有限或绝对值超过 1e90 时放弃证书，避免中间量溢出。

运动端点凸包的投影分离

Bernstein 无法判定时，尝试由端点边和法线构造的有限候选轴。运动几何体始终位于其端点凸包内；若任一轴上两个凸包的投影包围区间严格分离，就可证明整段轨迹不相交。轴的构造允许舍入，因为任意有限固定轴都可作为证书；投影点积本身必须向外包围。未找到分离轴不会被当作碰撞证明。

误差界的立方计算

C19 保留原来的非负系数 filter 和尺度 delta∈[0,1]，将通用 pow(delta,3) 改为逐项向上舍入乘法。设 up 为向正无穷取相邻 double，fl 为当前舍入模式下的乘法结果：

```
s = up(fl(delta * delta))
c = up(fl(s * delta))
e = up(fl(filter * c)) >= filter * delta^3
```

非负性使每一步上界可向后传播，范围约束避免溢出，渐进下溢由向外扩展覆盖。这一计算不声称与原 pow 逐位相同；实际编译函数已通过 16,384 项精确有理数上界检查，且最终保留碰撞输出的逐位检查通过 [S5]。

最终六轮均记录最近舍入、FTZ=0、DAZ=0，MXCSR=8064；未开启 fast-math。推广到不同浮点模式或编译器前，需要重新检查这些前提。

5 实验取舍与当前剩余瓶颈

哪些尝试没有按最初方案保留

候选	观测结果	当前版本中的处理
C08	子集 19.989 秒，但全量 67.502 秒	保留数学证书，改为 C09 的延迟调用
C10	模板回调单轮子集 20.456 秒，较父版本变慢	与 C11 内联合并后继续保留，不能声称其单项有益
C12	显式四通道 AVX2 子集 19.639 秒，四组均变慢	回退该实现，使用专用标量角点评估
C17	八个闭时间区间投影子集 19.579 秒；多去掉 11 个误报	额外计算成本过高，回退；浮点环境核验保留
C18	仅计算可细分方向的比值，子集 18.036 秒	未证明收益，恢复原 Eigen 选择表达式

固定筛选集为 546,337 条，包含全部人工查询、全部仿真真碰撞及仿真文件每 128 条取样。单轮子集仅用于筛选；不能把表中跨阶段时间当成同期独立消融。保留 / 淘汰记录及全部 20 个候选见 [S6]。

现有证据指向困难查询的搜索成本

C09 的历史诊断 [S7] 记录了 256 条超过 1 ms 的子集查询，合计 19.151 秒，其中真碰撞 6.984 秒，真值阴性 12.167 秒。这帮助定位了长时间搜索的查询；它是旧版本诊断，不能直接当作 C20 的热点占比。C20 的确定证据是人工查询仍占约一半的全量时间，见第 2 页。

当前已经减少重复算术、内存开销和部分无效搜索；但保留下来的真碰撞、近退化情况和证书无法排除的阴性查询，仍需原 BFS 细分。其各自耗时比例尚未在 C20 上重新做完整剖析。

距离 80% 目标还有多远

按本次原版中位数，80% 目标要求降至 14.549 秒以内；C20 仍需再降低约 55.90% 的当前耗时。仅将剩余小函数进一步内联能否达到这一量级，没有实验依据。

若另行开展下一轮研究，先在 C20 上补充分组与逐查询剖析，再对角点评估、证书、误差界和队列复用做受控消融。对困难查询尝试减少搜索规模的方案时，仍需保守性论证和同期全量验证。本报告未运行新的候选，也未将这些方向计入已取得的收益。

6 代码定位与复现依据

下列行号对应核心提交 0c1f229b4e10。当前工作树 src/ 与该提交一致。源码共涉及八个核心文件，其中三个为新增辅助头文件；大块 diff 包含格式调整，行数不代表算法改动规模。

代码位置	当前优化内容
src/tight_inclusion/corner_evaluation.hpp:13	时间与空间插值复用，精确二进制倒数
src/tight_inclusion/ccd.cpp:14,191,213,226,233	AABB、误差输入、初始盒前移和 VF/EE 顺序
src/tight_inclusion/interval.hpp:14,28,111	整数函数内联，NumCCD 紧凑布局，区间操作内联
src/tight_inclusion/interval_root_finder.cpp:15,159,203	队列包装、固定初始数组、模板细分回调
src/tight_inclusion/interval_root_finder.cpp:417,471,648	线程局部容量、延迟证书、误差界实现
src/tight_inclusion/bernstein_filter.hpp:11,35	向外舍入区间与共面性证书
src/tight_inclusion/projection_filter.hpp:7,25	投影区间和候选分离轴
interval.cpp 与 interval_root_finder.hpp	实现移入头文件及辅助接口声明

结果和证据索引

[S1] research-results/final80-c20-summary.json：最终六轮统计；runs/final80-c20-*/ 保存每轮日志、程序和源码快照。
[S2] research-results/final80-completion-audit.json：20 个候选、源码及缓存哈希审计。
[S3] research-results/c20-analysis-metrics.json：本报告的分组中位数与占比。
[S4] research/NUMERICS.rst 和 CONTINUED_NUMERICS.rst：等价性与保守性依据。
[S5] research-results/error-bound-audit/result.json：误差上界精确检查。
[S6] research/FINAL_RESULTS80.rst 及 research-results/cNN-experiment.json：完整实验沿革、取舍和复现说明。

[S7] research-results/diagnostics/c09-hotspots/summary.json：C09 历史逐查询诊断。

复核已保存的结果

在研究工作树根目录执行下列命令，仅复算统计，不启动 CCD：

```
python research/research.py summarize --prefix final80-c20 --target-reduction 0.80
```

重新进行同期全量实验

使用已冻结的最佳程序；选择尚不存在的输出 prefix。两版各三次交替运行：

```
python research/research.py paired --prefix c20-report-repro-01 --candidate-snapshot
research-results/runs/c20-subset --allow-fp-removal --target-reduction 0.80
```

工作树：D:/ly/DGP-Autoreasearch/Tight-Inclusion-autoresearch。最佳源码包为 research-results/best-c20-source.zip，核心补丁为 best-c20-core.patch。原 baseline、数据与原结果保留。布局变化要求重编译调用方；现有速度结论限定于本机单线程、指定参数和数据，未验证其他架构、多线程或不同 minimum_separation 的性能。